

AI-Powered Customer Support Assistant Demo Script

Build a Multi-Agent Customer Support Experience with AWS

Demo Deep Dive Doc

USE CASE: CUSTOMER SUPPORT ASSISTANT

User Story: A large retailer is rapidly scaling and faces increasing customer demand for fast, effective support. Customers expect seamless assistance with tasks like troubleshooting, personalized product recommendations, and order management. To meet these demands, the retailer needs a scalable, AI-powered solution. By leveraging Amazon Bedrock’s multi-agent collaboration, the retailer can efficiently manage high volumes of inquiries, provide personalized and insightful assistance, and free up human agents to handle more complex issues — ensuring a superior customer experience and operational efficiency.

Overview: As the retailer grows, so does the need for a robust customer support system capable of handling diverse and complex customer inquiries. A multi-agent solution built on Amazon Bedrock streamlines order management, resolves technical issues, offers tailored product recommendations, and secures customer interactions. By utilizing specialized agents, the system ensures faster, more accurate responses while scaling efficiently to meet increasing customer demands. Each agent is designed to address a specific task, working together seamlessly to deliver a high-quality customer experience.

In this demo, that multi-agent solution is built on **Amazon Bedrock AgentCore** and the **Strands Agents SDK**. Rather than a single monolithic chatbot, the assistant is a team of specialized agents coordinated by a Supervisor orchestrator. Each agent runs on a distinct Amazon Bedrock model (a deliberate multi-model mix), calls governed tools through a secure gateway, and streams its reasoning back to the user in real time — reading structured business data, searching unstructured knowledge, remembering the customer across sessions, and showing its work, all while content-safety and quality gates run on every turn.

This document walks through the five capabilities that make up the demo. For each step, use the listed prompt examples to drive the narrative live in the app.

STEP ONE: BUILDING THE MULTI-AGENT FOUNDATION

The support team needs to handle a wide range of requests — order status, product recommendations, technical troubleshooting, and personalized service — without building one brittle, do-everything bot. To do this, they host a multi-agent application on **Amazon Bedrock AgentCore Runtime**, a serverless, per-session isolated container environment, and build the agents with the **Strands Agents SDK** using the “agents as tools” pattern.

A **Supervisor orchestrator** (Amazon Nova Pro) analyzes each customer request, decomposes it into sub-questions, and routes those to the right specialists:

- **Order Management** (Claude Haiku 4.5) — order status, shipping, inventory availability
- **Product Recommendation** (Amazon Nova 2 Lite) — tailored product suggestions from purchase history and catalog
- **Troubleshoot** (Amazon Nova Micro) — technical resolution from FAQs and troubleshooting guides
- **Personalization** (Claude Sonnet) — customer preferences and cross-session context

Each specialist is exposed to the Supervisor as a tool with failure isolation, so one specialist being unavailable never breaks the rest of the response. The Supervisor consolidates the specialists’ output into a single, coherent answer. Using a distinct model per agent lets the demo match model capability and cost to each task — a lightweight model for narrow lookups, a stronger model for nuanced reasoning.

Real-world value: Multi-agent orchestration lets teams add, swap, or upgrade a specialist (or its model) without touching the others, and keeps each agent’s prompt and toolset focused — which improves answer quality and controls cost. *(Insert relevant customer reference.)*

STEP TWO: SECURE TOOL ACCESS AND THE DATA FOUNDATION

Great answers require real data. The specialists never touch data stores directly — instead they call tools published through **Amazon Bedrock AgentCore Gateway**, which exposes Lambda-backed tools as Model Context Protocol (MCP) endpoints secured with Cognito OAuth2 machine-to-machine auth. Two tools power the demo:

1. **athena_query** — read-only SQL over the retailer’s **structured** data (orders, inventory, product catalog, purchase history, and customer profiles), served by **Amazon Athena** using the **AWS Glue Data Catalog** over data in **Amazon S3** — no data movement required.
2. **kb_retrieve** — semantic search over the retailer’s **unstructured** content (customer feedback, product reviews, FAQs, troubleshooting guides), backed by three **Amazon S3 Vectors** knowledge bases (personalization, product, troubleshooting).

Because tool access is centralized in the Gateway, each agent gets exactly the tools it needs (Product Recommendation gets Athena + the product KB + a code sandbox; Troubleshoot gets only its KB), access is authenticated and auditable, and the model’s interface stays simple — the agent just calls **athena_query(query)** or **kb_retrieve(query)**.

Real-world value: Governed, centralized tool access means data sources stay behind authentication and least-privilege controls while any number of agents reuse them safely. *(Insert relevant customer reference.)*

STEP THREE: NATURAL LANGUAGE SUPPORT IN ACTION

This is the core of the live demo. Open the application and start on the **Chat** tab. Pick one of the sample questions to kick things off, then watch the **Agentic Workflow** diagram on the right light up as the Supervisor routes to specialists.

As the turn runs, expand the per-agent dropdowns (or click an agent node in the flowchart) to view the **Agent Traces** — the normalized, step-by-step record of what each agent did: its routing decision, reasoning, gateway tool calls, and knowledge base retrievals. The assistant’s answer streams into the chat in real time, and each specialist’s trace shows the tool call and the data it retrieved, so the audience can see exactly how the final answer was assembled.

When the turn is done, switch to the **Data** tab to review the underlying tables each agent has access to — a great way to compare the assistant’s answer against the source data. Tabs stay loaded as you switch, so you can move between Chat and Data without losing state.

Prompt examples (each exercises multiple agents):

- “I like good quality nova phones. Provide me a product suggestion, followed up with comments people say about it? - customer id cust010” — Product Recommendation + customer feedback KB
- * “I’m really interested in smart type watches. I don’t care about cost. Let me know what you have that you think I’d like. Make sure to let me know of any known issues and warranty information too. - customer id cust002” * — Product Recommendation + Troubleshoot
- “I ordered a promax laptop, and would like to know if it has been delivered yet. I also have been looking through some of your phones online. Recommend me a phone I’d like. - customer id cust005” — Order Management + Product Recommendation
- “Are there any available speakers in stock that I would like under \$100? And in case I run into issues with the product, give me some troubleshooting tips. - customer id cust007” — Product Recommendation + Troubleshoot + FAQ

Real-world value: A single natural-language front door routes to the right specialist automatically, so customers (and support agents) get consolidated answers without knowing which system holds which data. *(Insert relevant customer reference.)*

STEP FOUR: PERSONALIZATION AND MEMORY

Support should feel continuous, not like starting over every time. The assistant uses **Amazon Bedrock AgentCore Memory** for both short-term (within a session) conversation memory and long-term user-preference and semantic strategies that the **Personalization** agent draws on for cross-session recall.

Short-term memory is on by default and can be toggled per conversation from the tab row. Open the **Memory** tab to view the caller’s conversation history, read directly from AgentCore Memory and keyed to their authenticated identity. When memory is active, the Personalization agent can tailor recommendations and responses using what it already knows about the customer — preferences, past interactions, and behavior patterns — layered on top of the structured profile data it pulls from Athena.

Prompt examples:

- Ask a preference-laden question, approve/observe the answer, then ask a **follow-up** in the same session to show short-term recall (e.g., “*Actually, show me something in a different color.*”).
- Toggle short-term memory off and re-ask to contrast the stateless behavior.

Real-world value: Persistent, identity-scoped memory turns disconnected chats into an ongoing relationship, improving personalization and reducing repeat questions. *(Insert relevant customer reference.)*

STEP FIVE: GOVERNANCE, OBSERVABILITY, AND TRUST

Enterprises can’t ship an agent they can’t see or control. Every turn passes through two governance gates, and every step is captured for observability.

- **Guardrail (policy) check** — before the response is returned, a content-policy guardrail screens the turn for content safety (denied topics), PII, and prompt-attack patterns. The outcome (passed / redacted / blocked) is emitted to the trace stream and shown as the **Guardrail** node in the workflow diagram.
- **Evaluation** — each final response is automatically scored for relevance, completeness, and safety, producing an overall quality verdict (pass / review). Scores appear in the trace stream and as the **Evaluation** node — an inline quality gate on agent output.
- **Observability** — **AWS X-Ray** tracing with **Amazon CloudWatch** Transaction Search captures each agent step. That normalized trace stream (routing decisions, tool calls, KB retrievals, code runs, governance outcomes) is exactly what powers the Agent Traces view and the animated workflow diagram the audience has been watching throughout the demo.

To showcase this, run any prompt and point out the **Guardrail** and **Evaluation** nodes lighting up at the end of the turn, then open a trace dropdown to show the captured steps.

Real-world value: Built-in content safety, automated response evaluation, and end-to-end tracing make agent behavior auditable and trustworthy — the difference between a prototype and a production deployment. *(Insert relevant customer reference.)*

CONCLUSION

By combining Amazon Bedrock AgentCore (Runtime, Gateway, Memory, and Observability), the Strands Agents SDK, a multi-model mix, Amazon Athena over the Glue Data Catalog, and Amazon S3 Vectors knowledge bases, the retailer transforms customer support from a single brittle bot into a coordinated team of specialized, governed, and observable agents. Customers get faster, more accurate, and more personalized help; the support organization gets a system that is secure, auditable, and straightforward to extend — scaling quality support without scaling headcount.

LINKS

- **Application URL:** <https://d3h33pm6rwmsdm.cloudfront.net/>
- **Username:** aws_user
- **Password:** AWS123xyz!

This is a demonstration tool. Responses are AI-generated and may be inaccurate or incomplete. It runs on sample data and is not connected to real customer accounts, orders, or systems. Do not enter real personal, financial, or otherwise sensitive information.